GLM (Tsinghua University). GLM (130B) (Zeng et al., 2022) is a bidirectional Transformer language model trained simultaneously on English (the Pile Gao et al., 2021a), Chinese (several undisclosed corpora) and multi-task data from Sanh et al. (2021) and Wang et al. (2022a) (which only accounts for 5% of tokens) to improve zero-shot generalization. [116] The training procedure is based on GLM (Du et al., 2022). The release date is based on `https://keg.cs.tsinghua.edu.cn/glm-130b/posts/glm-130b/` and the model size for this model is based on Zeng et al. (2022).

YaLM (Yandex). YaLM (100B) is an autoregressive Transformer language model trained on 1.7TB of undisclosed data across both English and Russian comprised of both the Pile (Gao et al., 2021a) and several Russian corpora (predominantly derived from the Yandex Search index).[117] The release date and model size for this model are based on `https://github.com/yandex/YaLM-100B`.

J Adaptation

Here we provide the full details on how we adapt language models to fully concretize what we mean by 5-shot prompting.

J.1 Formatting test instances

Language modeling. For *language modeling* scenarios, the prompt is simply the input and there is no reference. Since documents in language modeling datasets are longer than the window size of the models we study, we tokenize documents using each model's corresponding tokenizer and chunk the resulting token sequences according to the model's window size. Following Gao et al. (2021a), we compute the perplexity metrics of each document separately and maximize the amount of context for each prediction by using as many tokens from the previous chunk as conditioning tokens as possible.

Truncation. As we describe for language modeling scenarios, it is possible that test instances for any scenario are longer than a model's window size. For such instances, rather than chunking them as we do in language modeling, we truncate them so that they fit within the model's context window.

Multiple choice. For *multiple choice* scenarios, each instance has several answer choices as references (usually with one marked as correct). In traditional approach to classification (e.g. a CNN trained to perform image classification on ImageNet (Deng et al., 2009)), a model would explicitly predict a distribution over answer choices (and, generally, the argmax of this distribution would be the model's prediction). However, given the flexibility of the natural language interface of a language model, multiple choice classification can be implemented in several ways.

Prior work introduces two classes of approaches to address adaptation for multiple choice scenarios. The first is the *separate* approach employed by Brown et al. (2020), where each each choice each choice is independently scored by concatenating it with the question and computing its probability according to the LM. In this approach, the probabilities may be further *calibrated* by the probability the LM assigns to the answer choice alone. The second is the joint approach employed by Hendrycks et al. (2021c), where all the choices are concatenated with the question to form a prompt (e.g., "`<question> A. <choice 1> B. <choice 2> Answer:`") and the LM predicts the choice index (e.g., `A` or `B`). This resembles how one might see a multiple choice question presented on an exam or test.

In §8.2: PROMPTING-ANALYSIS, we conduct ablations to study how the choice of adaptation method for multiple choice scenarios influences performance, finding the choice leads to substantial changes in accuracy (among other metrics). For each multiple choice scenario, we provide a default adaptation method. To select this default, we follow prior work if prior work establishes one method as preferable. Otherwise, we default to the joint approach unless the answer choices are natural continuations of the input (e.g. **HellaSwag**), in which case we use the separate approach. We prefer the multiple choice approach for three reasons: (i)

[116] `http://keg.cs.tsinghua.edu.cn/glm-130b/posts/glm-130b/`
[117] `https://github.com/yandex/YaLM-100B`

it requires one query per question as opposed to one query per answer choice, (ii) it enables the use of in-context training examples to improve performance, and (iii) it facilitates direct comparison of choices for the model, which handles choices like 'none of the above' in a more sensible way.

J.2 Formatting the remainder of the prompt

Selection of training examples. For all non-language modeling scenarios, we include training examples in-context to specify the desired model behavior.[118] We select 5 examples by default, though for scenarios with long examples, we adaptively reduce the number of examples based on the model's window size. Our selection policy for these examples emphasizes *class coverage*: for classification problems (i.e. where the output is from a closed set), we iterate over the possible classes in order of decreasing frequency, randomly sampling a representative for each class.

Critically, we *fix* these training examples for all instances to better reflect the realities of few-shot learning and the potential brittleness of this approach. This is contrast to the common practice of selecting a different set of instances for each test instance (Brown et al., 2020), which may lead to lower variance than would be seen in practice under true few-shot conditions (Perez et al., 2021). Since we expect the selection of training examples to be a significant contributor to variance, we repeat this process 3 times to measure the variance.

Prompt construction. In addition to the test instance and in-context training examples, we complete the prompt by providing instructions or prefixes that further delineate the desired model behavior. Prior work has extensively shown that the design of prompts matters (Le Scao & Rush, 2021; Liu et al., 2022b) with Bach et al. (2022) introducing a repository for prompts.[119]

Our philosophy is to view prompting as user behavior, consistent with our treatment of these models as *interfaces*: rather than having to optimize prompts to maximize model performance (Shin et al., 2020), we should strive for models that perform well on naturalistic prompts. Moving forward, we further expect that it may be desirable for language models to be *interoperable* meaning different models should be standardized such that they all work well for similar user prompting behavior. We encourage future work to explore how users are likely to prompt models in different circumstances, including how this depends on the broader interface that shapes the user's experience.

J.3 Decoding parameters

Once the test instance (§J.1: PROMPTING-TEST) and broader prompt (§J.2: PROMPTING-REMAINDER) are specified, the decoding parameters must also be specified to generate the model completion. These parameters are the temperature, stop condition (i.e. max tokens and stop sequences), and number of completions.

Temperature. The temperature modulates the distribution over next words used to decode text from the model: a temperature of 0 corresponds to selecting the argmax, of 1 corresponds to sampling from the distribution, and of ∞ corresponds to sampling from the uniform distribution. In recent years, alternative methods for decoding that post-process the model's distribution have been popularized like top-k sampling and nucleus sampling (Holtzman et al., 2020). Empirically, these methods generally improve the quality of model generations when models generate longer sequences.

In this work, since several of the models only support temperature-based sampling, as opposed to other sampling methods, we restricted ourselves to temperature-scaling during the sample process.[120] Consequently, for all scenarios where the ideal model behavior is to generate a very short sequence that should match the correct reference, we set the temperature to be zero as we desire the argmax under the model's distribution. For the longer-form generation scenarios such as text summarization, we either follow prior work or specify a process by which we arrived at the temperature we used.

[118] This is unnecessary for language modeling as the model by default produces a probability, which is the required behavior.
[119] `https://github.com/bigscience-workshop/promptsource`
[120] Note that it is possible to implement nucleus sampling *post hoc* even when a model provider does not natively support this, but this leads to a prohibitive increase in the number of tokens needed to query the model.

Stop Condition. Beyond the context window as a fundamental limit to the number of tokens a particular model can generate, we specify additional stop conditions to terminate decoding. The stop sequence(s) is a sequence of tokens that, if encountered during decoding, terminate the completion. Alternatively, max tokens is an absolute threshold on the number of tokens in the completion. In general, since we specify adaptation in a model-agnostic (and, hence, tokenizer-agnostic) fashion, we prefer to use stop sequence(s) as the (primary) stop condition to not make assumptions on the tokenizer (e.g. if model A uses a character-level tokenizer and model B uses a subword-level tokenizer, setting the same number of max tokens for both as the primary stop condition likely will lead to an unfair comparison). Since we provide in-context examples that we separate using a newline character, we generally set the stop sequence to be the newline character "`\n`". We find that across most instances and scenarios, model generation for all models terminates due to this condition. As a fail-safe to avoid the model generating an excessive number of tokens in the completion, we additionally set max tokens for each scenario based on the longest reference's length for that scenario.

Number of outputs. For a given prompt, since the model's completion involves sampling, there is randomness involved in determining the specific completion decoded for each instance. For most scenarios, since we are decoding the argmax through low temperature decoding, this is not a significant factor. However, for a few scenarios where we want to study the distribution of outputs for a given prompt, we sample multiple outputs per input. When relevant, we note this, though by default the number of outputs we sample per input is 1 (or 0 for language modeling as no output is generated).

J.4 Adaptation methods

Have specified our approach to adaptation via few-shot prompting, we now describe the specific adaptation methods. We discuss these implicitly above, but we make explicit in Table 14 how each scenario is associated with an adaptation method by default that dictates how the scenario is addressed by the language model. This relates to the experiments in §8.2: PROMPTING-ANALYSIS.

Adaptation method	Scenarios
Language modeling	**The Pile**, **ICE**, **TwitterAAE**
Multiple choice (joint)	**MMLU**, **TruthfulQA**, **LegalSupport**, **LSAT**, **BBQ**
Multiple choice (separate)	**BLiMP**
Multiple choice (separate-calibrated)	
Generation	**BoolQ**, **NaturalQuestions** (open-book), **NaturalQuestions** (closed-book), **NarrativeQA**
	QuAC, **XSUM**, **CNN/DailyMail**, **IMDB**, **CivilComments**
	RAFT, **WikiFact**, synthetic reasoning, synthetic reasoning (natural)
	bAbI, Dyck, **GSM8K**, **MATH**, **MATH** (chain-of-thoughts)
	HumanEval, **APPS**, **EntityMatching**, **DataImputation**
	Copyright (text), **Copyright** (code), disinformation (reiteration), disinformation (wedging)
	BOLD, **RealToxicityPrompts**
Ranking	**MS MARCO (regular)**, **MS MARCO (TREC)**

Table 14: **Default adaptation methods.** For each adaptation method, we specify the scenarios that use the method by default. We do not specify defaults for **HellaSwag** and **OpenBookQA** currently.